COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	8/4/2026	**DATE SUBMITTED:**	8/11/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 5 QUEUES

1. PROCEDURE OUTPUT

ILO A:
Main Code

main.cpp ...\ILO A | main.cpp ...\ILO B | ostream.h | que

Dimaguiba_HOA5P1 > Queue > ILO A > main.cpp > ...

```cpp
#include<iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    //create an object
    std::queue<char> myQ;
    myQ.push('J');
    myQ.push('A');
    myQ.push('M');
    myQ.push('E');
    myQ.push('S');
    display(myQ);

    //try all member functions:
    //empty
    std::cout<<myQ.empty()<<std::endl;
    //size
    std::cout<<myQ.size()<<std::endl;
    //front
    std::cout<<myQ.front()<<std::endl;
    //back
    std::cout<<myQ.back()<<std::endl;
    //pop
    std::cout<<"Popping: "<<myQ.front()<<std::endl;
    myQ.pop();
    //display
    display(myQ);
    //push
    std::cout<<"Pushing: J"<<std::endl;
    myQ.push('J');
    //display
    display(myQ);
    return 0;
}

void display(std::queue<char> copyQ){
    //create copy of queue that was passed
    std::queue<char> c = copyQ;
    while(!c.empty()){
        std::cout<<" "<<c.front();
        c.pop();
    }
    std::cout<<"\n";
}
```

Output:

```
derr=Microsoft-MI
 J A M E S
0
5
J
S
Popping: J
 A M E S
Pushing: J
 A M E S J
```

Observations:

The code makes use of the Queue template library (STL) with the following commands. Similar to stack, the only difference is that the FIRST element will be the first to leave as well (FIFO). The push function adds or inserts an element to the queue, the pop function removes the FRONT of the list (aka the first element added). The front function is self-explanatory in that they return the front element. The size returns size, and empty returns true or false whether it's empty or not. The display function makes a copy of the current queue, puts it into a temporary variable, displays the front, then pops to display the next one.

ILO B:

Main:

Dimaguiba_HOA5P1 > Queue > ILO B > main.cpp > ...

```cpp
#include <iostream>
#include "queueLL.h"

int main(){

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('J', &front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;
    enqueue('A', &front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;
    enqueue('M', &front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;
    enqueue('E', &front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;
    enqueue('S', &front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;

    //display all:
    displayAll(front);

    //dequeue
    dequeue(&front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;
    dequeue(&front, &back); //should be empty

    enqueue('J', &front, &back);
    getFront(front);
    enqueue('A', &front, &back);
    getBack(back);
    std::cout<<front-> data<<" "<<back->data<<std::endl;

    //remove again
    dequeue(&front, &back);
    dequeue(&front, &back);

    //empty check
    getBack(back);
    getFront(front);
    displayAll(front);
    return 0;
}
```

Header File:

Dimaguiba_HOA5P1 > Queue > ILO 8 > C queueLL.h > ...

```
#ifndef QUEUELL_H
#define QUEUELL_H

//create the class node
template <typename T>
class qNode{
    public:
    T data;
    qNode<T>* next;
};

//creates the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//insert/enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create new node
    qNode<T>* newNode = new_node(newData);

    //insert into empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
    //insert an item into non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;

}

//delete/dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){

    //create temporary mode
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    //add a check if queue is already empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"The queue is empty!"<<std::endl;
        return;
    }

    //check if the queue has only 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }
    //delete the node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;
}

//check the front
template<typename T>
void getFront(qNode<T>* frontPtr){
    if (frontPtr == nullptr){
        std::cout<<"Queue is empty!"<<std::endl;
        return;
    }

    std::cout<<"The current front: "<<frontPtr->data<<std::endl;

}

//check the back
template<typename T>
void getBack(qNode<T>* backPtr){
    if (backPtr == nullptr){
        std::cout<<"Queue is empty!"<<std::endl;
        return;
    }

    std::cout<<"The current back: "<<backPtr->data<<std::endl;

}

//display all
template <typename T>
void displayAll(qNode<T>* frontPtr){
    if(frontPtr == nullptr){
        std::cout<<"The queue is empty!"<<std::endl;
        return;
    }
    std::cout<<"The current Queue: ";
    while(frontPtr != nullptr){
        std::cout<<frontPtr->data<<" ";
        frontPtr = frontPtr->next;
    }
    std::cout<<"\n";

}


#endif
```

OUTPUT:

```
PS C:\Users\TIPQC\Documents\Dimags>  & 'c:\Use
derr=Microsoft-MIEngine-Error-jljuztts.23x' '-
J J
J A
J M
J E
J S
The current Queue: J A M E S
A S
M S
S S
The current front: J
The current back: A
J A
Queue is empty!
Queue is empty!
The queue is empty!
PS C:\Users\TIPQC\Documents\Dimags>
```

Observation:

The code is made up of a main driver file and a header file to hold all the methods. It makes use of a linked-list. The same functions before are present in the header files with only some differences. First, the front and back nodes are defined and are connected to nothing, the enqueue function creates a new node in which if it were the first node, points both the front and the back to it; otherwise, it inserts it into the back of the list, changing the address of the back to it. Dequeue is the complete opposite, this time, REMOVES the front and sets the front's address to the next. getFront works the same, takes the front element of the queue; so does getBack. The displayAll function takes the address of front and traverses till it reaches the back, displaying the entire queue.

ILO B2:

CODE
Main:

```cpp
#include <iostream>
#include "queueArr.h"

int main(){
    //create queue
    queueArr<char> q(6);

    //Enqueue
    q.Enqueue('J');
    q.Enqueue('A');
    q.Enqueue('M');
    q.Enqueue('E');
    q.Enqueue('S');

    //check front and back
    std::cout<<q.Front()<<std::endl;
    std::cout<<q.Back()<<std::endl;

    //Dequeue
    q.Dequeue();
    q.Dequeue();

    //check front and back
    std::cout<<q.Front()<<std::endl;
    std::cout<<q.Back()<<std::endl;
    //copy
    queueArr<char> qTPi = q; //"Que - T - Pi haha"
    std::cout<<qTPi.Front()<<std::endl;
    qTPi.Back();

    //return 1/0 if empty or full
    q.Clear();
    std::cout<<q.Full()<<std::endl;
    q.Empty();
    return 0;
}
```

Header file:

VSCODES > ARRAYQUEUE > C queueArr.h > queueArr<T> > ~queueArr()

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>


template <typename T>
class queueArr{
    private:
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        queueArr(size_t capacity);
        ~queueArr();

        bool Empty();
        bool Full();

        size_t Size();

        void Clear();

        void Enqueue(T newData);
        void Dequeue();

        T Front();
        T Back();

        queueArr(const queueArr &other);
        queueArr &operator = (const queueArr &other);

};

template <typename T>
queueArr<T>::queueArr(size_t capacity){
    //define capacity of array
    q_capacity = capacity;
    //set the size to 0
    q_size = 0;
    //set the front and back to 0 and -1
    q_front = 0;
    q_back = -1;
    //allocate a new space for the q_array that was created
    q_array = new T[q_capacity];
}

```

```cpp

template <typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
    std::cout<<"destructor was called\n";
}

template <typename T>
queueArr<T>::queueArr(const queueArr &other){
        std::cout<<"Queue Copied!"<<std::endl;
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
        //create a new array to avoid pointing to the
        q_array = new T[q_capacity];
        for(int i = 0; i < q_capacity; i++){
            q_array[i] = other.q_array[i];
        }
    }


template <typename T>
bool queueArr<T>::Empty(){
    return q_size == 0;
}

template <typename T>
bool queueArr<T>::Full(){
    return q_size == q_capacity;

}

template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

template <typename T>
void queueArr<T>::Clear(){
    q_front = 0;
    q_back = -1;
    q_size = 0;

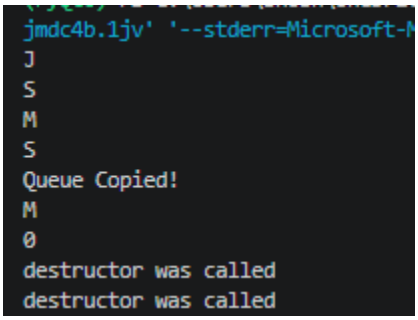
    return;
}

template <typename T>
T queueArr<T>::Front(){

    //check if queue is not empty
    if(Empty()){
        std::cout<<"The queue is empty"<<std::endl;
        return T{};//std::optional
    }
    return q_array[q_front];
}
```

```
108
109 template <typename T>
110 T queueArr<T>::Back(){
111
112     //check if queue is not empty
113     if(Empty()){
114         std::cout<<"The queue is empty"<<std::endl;
115         return T{};//std::optional
116     }
117     return q_array[q_back];
118 }
119
120 template <typename T>
121 void queueArr<T>::Enqueue(T newData){
122     //check if full
123     if(Full()){
124         std::cout<<"queue is full!"<<std::endl;
125         return;
126     }
127     //compute for back index
128     q_back = (q_back + 1) % q_capacity;
129     //store
130     q_array[q_back] = newData;
131     //increment size
132     q_size++;
133 }
134
135 template <typename T>
136 void queueArr<T>::Dequeue(){
137     //check if full
138     if(Empty()){
139         std::cout<<"queue is empty!"<<std::endl;
140         return;
141     }
142     //compute for back index
143     q_front = (q_front + 1) % q_capacity;
144     //decrement size
145     q_size--;
146
147     //what if empty?
148     if (q_size == 0){
149         Clear();
150         return;
151     }
152 }
153
154 #endif
```

OUTPUT



Observation:

The queue is all bottled into one class. Before constructions, a set of values are created. The size, the max capacity, the array itself, and both front and back. These values get modified as the functions are used. When constructing, since the array is connected to a pointer, copying the pointer wouldn't really be a new array. That's why the copy constructors make use of a loop and a new pointer to copy elements. When enqueueing or dequeueing, it takes the current index, adds onto it then inserts the element, otherwise the opposite happens. The modulo (%) is so that it wraps around the array when the end is reached, like a circle or a ring. Finally, the front and back do what they are named and return the front and the back.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE

Main:

VSCODES > Dimaguiba_Queue > ILO C > main.cpp > ...

```cpp
#include <iostream>
#include <string>
#include "queueLL.h"

class Job{
    public:
        int id, pages;
        std::string username;

        Job(int idNo = 0, int pageNo = 0, std::string userName = ""){
            id = idNo;
            pages = pageNo;
            username = userName;
        }

};
    Printer<Job>* front = nullptr;
    Printer<Job>* back =  nullptr;

```

```cpp
int option, pages, id;
std::string username;

int main(){
    while (option !=  6){
    std::cout<<"   <A \"Printer\">\nSelect an option:"<<std::endl;
    std::cout<<"1. Print All\n2. Queue a File\n3. View Next\n4. View Last \n5. View Queue\n";
    std::cout<<"6. Quit"<<std::endl;
    std::cout<<"Enter Choice here: ";

    while(!(std::cin>>option)){
        std::cout<<"Invalid input!\n";
        std::cin.clear();
        std::cin.ignore(10000, '\n');
    }

   switch(option){
     case 1:
       while(front != nullptr){
        dequeue(&front, &back);
       }
        break;

     case 2:
     {
       std::cout<<"ID Number: ";
       std::cin >> id;

       std::cout<<"Number of Pages: ";
       std::cin>>pages;

       std::cout<<"Enter Username: ";
       std::cin.ignore();
       std::getline(std::cin, username);

       Job j(id,pages,username);
       enqueue(j, &front, &back);
       break;
     }

     case 3:
     {
        getFront(front);
        break;
     }

     case 4:
     getBack(back);
     break;

     case 5:
     displayAll(front);
     break;

     case 6:
     break;

     default:
        std::cout<<"Invalid Choice\n";

     }

   }
   return 0;
}
```

Header:

VSCODES > Dimaguiba_Queue > ILO C > C queueLL.h > dequeue<T>(Printer<T>**, Printer<T>**)

```
#ifndef QUEUELL_H
#define QUEUELL_H


//create the class node
template <typename T>
class Printer{
    public:
    T data;
    Printer<T>* next;
};


//insert/enqueue
template <typename T>
void enqueue(T newData, Printer<T>** frontPtr, Printer<T>** backPtr){
    //create new node
    Printer<T>* newNode = new Printer<T>;
    newNode->data = newData;
    newNode->next = nullptr;

    //insert into empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        std::cout<<"Queued: "<<std::endl;
        std::cout<<"ID: "<<(*backPtr)->data.id<<" ";
        std::cout<<"pages: "<<(*backPtr)->data.pages<<" ";
        std::cout<<"Username: "<<(*backPtr)->data.username<<std::endl;
        return;
    }

    //insert an item into non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;

    std::cout<<"Queued: "<<std::endl;
    std::cout<<"ID: "<<(*backPtr)->data.id<<" ";
    std::cout<<"pages: "<<(*backPtr)->data.pages<<" ";
    std::cout<<"Username: "<<(*backPtr)->data.username<<std::endl;
}

//delete/dequeue
template <typename T>
void dequeue(Printer<T>** frontPtr, Printer<T>** backPtr){

    //create temporary mode
    Printer<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    //add a check if queue is already empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"The queue is empty!"<<std::endl;
        return;
    }
    std::cout<<"Now Printing: "<<std::endl;
    std::cout<<"ID: "<<(*frontPtr)->data.id<<" ";
    std::cout<<"pages: "<<(*frontPtr)->data.pages<<" ";
    std::cout<<"Username: "<<(*frontPtr)->data.username<<std::endl;

    //check if the queue has only 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }
    //delete the node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;
```

```
72  }
73
74  //check the front
75  template<typename T>
76  void getFront(Printer<T>* frontPtr){
77      if (frontPtr == nullptr){
78          std::cout<<"Queue is empty!"<<std::endl;
79          return;
80      }
81
82      std::cout<<"Next in Queue: "<<std::endl;
83      std::cout<<"ID: "<<frontPtr->data.id<<" ";
84      std::cout<<"pages: "<<frontPtr->data.pages<<" ";
85      std::cout<<"Username: "<<frontPtr->data.username<<std::endl;
86
87  }
88
89  //check the back
90  template<typename T>
91  void getBack(Printer<T>* backPtr){
92      if (backPtr == nullptr){
93          std::cout<<"Queue is empty!"<<std::endl;
94          return;
95      }
96      std::cout<<"Last Queued: "<<std::endl;
97      std::cout<<"ID: "<<backPtr->data.id<<" ";
98      std::cout<<"pages: "<<backPtr->data.pages<<" ";
99      std::cout<<"Username: "<<backPtr->data.username<<std::endl;
100 }
101
102 //display all
103 template <typename T>
104 void displayAll(Printer<T>* frontPtr){
105     if(frontPtr == nullptr){
106         std::cout<<"The queue is empty!"<<std::endl;
107         return;
108     }
109     std::cout<<"The current Queue:"<<std::endl;
110     while(frontPtr != nullptr){
111         std::cout<<"ID: "<<frontPtr->data.id<<" ";
112         std::cout<<"pages: "<<frontPtr->data.pages<<" ";
113         std::cout<<"Username: "<<frontPtr->data.username<<std::endl;
114         frontPtr = frontPtr->next;
115     }
116     std::cout<<"\n";
117
118 }
119
120
121
122 #endif
```

OUTPUT

Menu:

```
   <A "Printer">
Select an option:
1. Print All
2. Queue a File
3. View Next
4. View Last
5. View Queue
6. Quit
Enter Choice here:
```

If empty:

```
Enter Choice here: 3
Queue is empty!
   <A "Printer">
Select an option:
1. Print All
2. Queue a File
3. View Next
4. View Last
5. View Queue
6. Quit
Enter Choice here: 4
Queue is empty!
```

Queueing and viewing:

```
   <A "Printer">
Select an option:
1. Print All
2. Queue a File
3. View Next
4. View Last
5. View Queue
6. Quit
Enter Choice here: 2
ID Number: 67
Number of Pages: 21
Enter Username: LJ
Queued:
ID: 67 pages: 21 Username: LJ
```

```
Enter Choice here: 3
Next in Queue:
ID: 67 pages: 21 Username: LJ
   <A "Printer">
Select an option:
1. Print All
2. Queue a File
3. View Next
4. View Last
5. View Queue
6. Quit
Enter Choice here: 4
Last Queued:
ID: 1234 pages: 999 Username: all ur ink gone lol
   <A "Printer">
```

```
Enter Choice here: 5
The current Queue:
ID: 67 pages: 21 Username: LJ
ID: 1234 pages: 999 Username: all ur ink gone lol
```

Print all:

```
Enter Choice here: 1
Now Printing:
ID: 67 pages: 21 Username: LJ
Now Printing:
ID: 1234 pages: 999 Username: all ur ink gone lol
```

Supplementary questions + Code Analysis:

The method or implementation used was through a linked list. This is due to its versatility and reusability. Also, by using headers and the template operations, the data type that the class can accept changes automatically with the input. Compared to arrays, the only limit of Linked-list is the computer's memory. With just a little bit of modifications to the code, inputting data from the Jobs class has never been easier as Linked lists allow for the input of several element types in a row. Just like the linked list from before, it features all queue operations required for it to function. In order to call on a class member, simply add a period or dot next to the member name in the data pointer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.